

Propuesta inicial

Sistema de Pedidos de Restaurante

Estado: primera propuesta (síntesis de análisis; aún sin implementación)

1. Qué queremos construir

Un **sistema de pedidos para restaurante** que permita gestionar el menú, abrir y editar comandas, respetar el estado de cocina, calcular totales de forma determinista y cerrar la orden cobrando por distintos medios.

Problema de negocio

Los restaurantes reciben pedidos por **varios canales** (mesa en sala, para llevar, delivery). Esos canales y las herramientas externas (BD, pasarelas, frameworks) cambian con frecuencia.

Si las reglas de venta (precios, descuentos, impuestos, edición según cocina, cobro) quedan atadas a un framework o a una base de datos concreta, **cada cambio de infraestructura rompe el negocio**.

Objetivo de la propuesta

Construir un sistema donde:

- Las **reglas de venta vivan en un núcleo estable**.
- Canales, UI, BD y cobros se conecten como **adaptadores**.
- Se cumplan los requisitos **RF1–RF5** de forma testeable y evolutiva.

2. Requisitos funcionales (análisis)

ID	Requisito	Qué implica en la propuesta
RF1	Administrar menú: platos, modificadores (extra/exclusiones), precios e impuestos	Catálogo de dominio + persistencia; UI de administración.
RF2	Abrir comanda/orden vinculada a mesa física o ID de pedido externo	Un mismo caso de uso <code>AbrirOrden</code> ; el origen (mesa / externo) es dato de canal, no lógica distinta.
RF3	Agregar, modificar o cancelar líneas validando estado de cocina; no editable tras iniciar cocción	Regla de dominio en el agregado <code>Orden</code> + máquina de estados.
RF4	Calcular subtotales, propinas opcionales, descuentos e impuestos de forma determinista	Cálculo en el núcleo (Money / Value Object + Strategy de descuento/impuesto); no en la UI.
RF5	Cerrar orden y registrar pago vía múltiples adaptadores (efectivo, tarjeta, pasarela)	Puerto de cobro + varias Strategies; mismo flujo de cierre.

Alcance de esta primera propuesta

Incluye

- Modelo de dominio y casos de uso para RF1–RF5.
- API REST + UI web.
- Persistencia en una base de datos.
- Cobros **simulados** (adaptadores fake listos para sustituir).

No incluye (fase posterior / fuera de MVP)

- Integraciones reales con apps de delivery de terceros.
- Pasarelas de pago en producción.
- Microservicios, mensajería event-driven como base, auth avanzada.
- App móvil nativa.

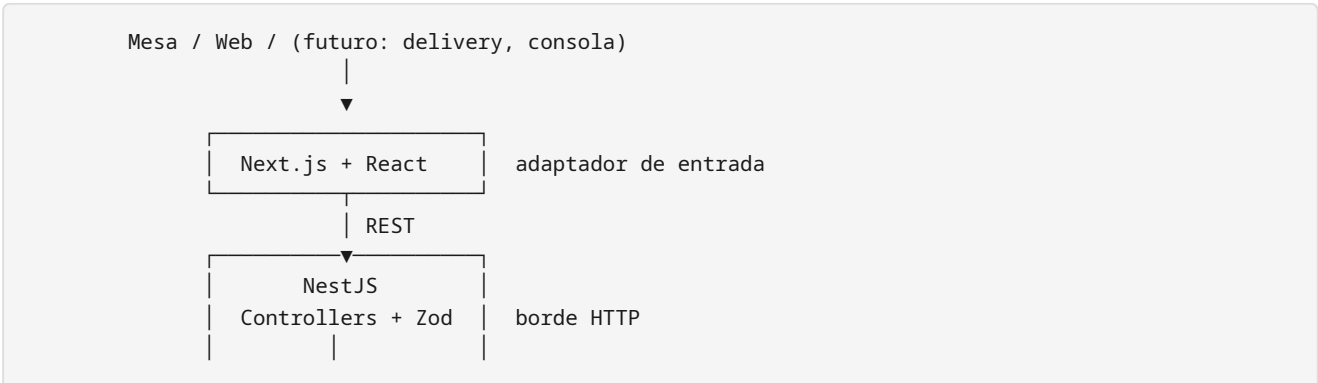
3. Decisiones ya tomadas (marco de la propuesta)

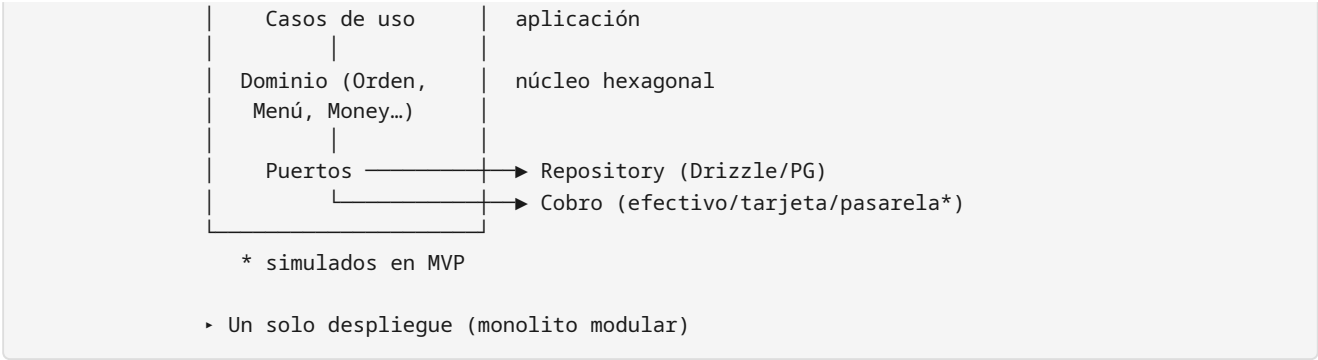
Nivel	Decisión
Diseño interno	Arquitectura Hexagonal (puertos y adaptadores); núcleo al estilo Clean (entidades + casos de uso)
Estilo de sistema	Monolito modular
Despliegue	Único (API + web + una BD)
Patrones clave	Aggregate, State, Strategy, Money/VO, Repository, Adapter, Use Case, DI, Factory si aplica
Stack	TypeScript, NestJS, PostgreSQL + Drizzle, Zod, pnpm, Next.js + React

Detalle en los documentos de análisis listados al inicio; esta propuesta **no los sustituye**, los consolida.

Construcción hexagonal: ver guía canónica `arquitectura-hexagonal.md` (cómo se estructuran puertos, adaptadores y carpetas).

4. Visión de la solución





Regla de dependencia: el dominio no importa Nest, Next, Drizzle ni SDKs de pago. Solo habla por puertos (interfaces).

5. Módulos del monolito

Módulo	Responsabilidad	RF
Menú	Platos, modificadores, precios, impuestos aplicables	RF1
Órdenes	Abrir comanda, líneas, vínculo mesa/ID externo	RF2, RF3
Cocina / estado	Transiciones de estado; bloqueo de edición	RF3
Cálculo	Subtotal, propina, descuentos, impuestos, total	RF4
Pagos	Cierre de orden y registro de cobro	RF5

Todos conviven en la misma aplicación Nest; las fronteras son lógicas (paquetes/módulos), no servicios de red.

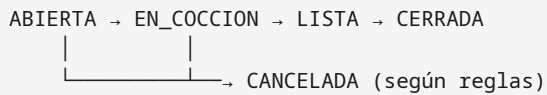
6. Modelo de dominio (borrador)

Entidades / agregados principales

Concepto	Rol
Plato	Ítem de menú con precio base e impuestos aplicables.
Modificador	Extra o exclusión asociada a un plato / línea.
Orden	Agregado raíz: origen (mesa o externo), líneas, estado, totales, pago.
LineaPedido	Plato + modificadores + cantidad + precios capturados.
Pago	Medio, monto, resultado del cobro.

Estados de la orden (State)

Propuesta inicial de ciclo de vida:



- En **ABIERTA** : se pueden agregar / modificar / cancelar líneas.
- Desde **EN_COCCION** : **líneas no editables** (RF3).
- **CERRADA** : tras cobro exitoso (RF5); sin más cambios de negocio.

Value objects

- **Money** (monto en unidad mínima + moneda)
- Identificadores de mesa / pedido externo
- Resultados de cálculo (**Subtotal** , **Impuesto** , **Propina** , **Total**)

7. Casos de uso (aplicación)

Caso de uso	RF	Notas
AdministrarPlato / CRUD menú y modificadores	RF1	Persistencia vía MenuRepository .
AbrirOrden	RF2	Recibe mesa o ID externo.
AgregarLinea / ModificarLinea / CancelarLinea	RF3	Fallan si el estado no permite edición.
IniciarCoccion (o transición de estado)	RF3	Pasa a EN_COCCION y bloquea edición.
CalcularTotales	RF4	Determinista; Strategies de descuento/impuesto.
CerrarOrden	RF5	Calcula si hace falta, cobra vía Strategy, persiste, cierra.

Los controllers Nest y las pantallas Next **solo disparan** estos casos de uso.

8. Puertos y adaptadores (propuesta)

Entrada (driving)

Adaptador	Tecnología	Fase
API HTTP	NestJS controllers + Zod	MVP
UI web	Next.js + React	MVP
Consola / delivery	—	Posterior

Salida (driven)

Puerto	Adaptador MVP	Sustituible por
MenuRepository / OrdenRepository	Drizzle + PostgreSQL	Otro ORM/BD sin tocar dominio
PaymentPort	Strategies: efectivo, tarjeta fake, pasarela fake	SDKs reales
(Opcional) Clock / config de impuestos	Implementación simple	Reglas fiscales más ricas

9. Stack y estructura de repositorio

Stack

1. TypeScript
2. NestJS
3. PostgreSQL + Drizzle
4. Zod
5. pnpm
6. Next.js + React

Estructura sugerida

```

Restaurante/
  docs/
  apps/
    api/      # NestJS – dominio, aplicación, infraestructura, HTTP
    web/      # Next.js – UI (adaptador)
  pnpm-workspace.yaml

```

Capas en `apps/api`:

```

src/
  domain/
  application/
  infrastructure/
  interface/    # HTTP + DTOs Zod

```

10. Flujos principales (MVP)

A. Pedido en mesa

1. Abrir orden con número de mesa.
2. Agregar líneas (platos + modificadores).
3. Enviar a cocina → estado `EN_COCCION` (ya no se editan líneas).
4. Calcular totales (propina/descuento opcionales).

5. Cerrar con pago (efectivo / tarjeta / pasarela simulada).

B. Pedido con identificador externo

Igual que A, pero `AbrirOrden` recibe un ID externo en lugar de mesa. Misma lógica de dominio.

C. Administración de menú

CRUD de platos y modificadores; precios e impuestos quedan disponibles para el cálculo (RF4).

11. Plan de entrega propuesto (fases)

Fase	Entrega	RF
0	Repo, docs, esqueleto Nest/Next, Postgres local	—
1	Dominio <code>Orden</code> + State + tests de edición/cocina	RF2, RF3
2	Menú + repositorios Drizzle	RF1
3	Cálculo determinista + Money + Strategies descuento/impuesto	RF4
4	Cierre + Strategies de cobro simuladas	RF5
5	UI Next: menú, comanda, cobro	RF1–RF5 (experiencia)

Criterio de avance: cada fase deja el núcleo **testable sin UI** cuando aplique.

12. Criterios de éxito de esta propuesta

La propuesta se considera viable si:

- RF1–RF5 están cubiertos por casos de uso del núcleo.
- Cambiar de BD o de medio de cobro no obliga a reescribir reglas de venta.
- La regla “no editable en cocción” se demuestra con tests de dominio.
- El cálculo de totales es reproducible con los mismos inputs.
- El sistema se despliega como monolito modular (API + web + una BD).

13. Riesgos y mitigaciones

Riesgo	Mitigación
Filtrar lógica de negocio a Next o a controllers	Casos de uso + tests de dominio obligatorios.
Usar <code>float</code> para dinero	Value Object <code>Money</code> desde el inicio.
Sobreingeniería (eventos, microservicios)	Apegarse al monolito modular y al set mínimo de patrones.
Cobros reales prematuros	Adaptadores simulados detrás de <code>PaymentPort</code> .

14. Conclusión

Propuesta: construir un **monolito modular hexagonal** en **TypeScript (NestJS + Next.js + PostgreSQL/Drizzle)** que aísle menú, órdenes, cocina, cálculo y pagos en un núcleo estable, conectando UI, persistencia y cobros como adaptadores, para cumplir **RF1–RF5** sin acoplar el negocio a la infraestructura.

Esta es la **primera propuesta** de trabajo. Los siguientes pasos naturales son: refinar el modelo de datos, definir contratos de API REST y arrancar la Fase 0 del plan de entrega.